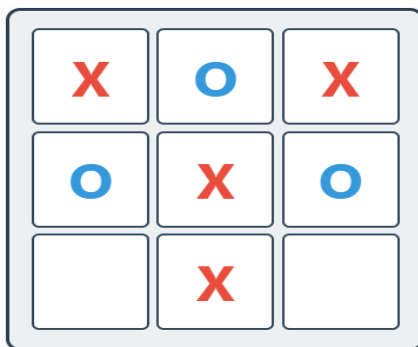




University of Asia Pacific

Artificial Intelligence and Expert Systems Lab

Course code: **CSE 404**



Assignment on Adversarial Search

Tic Tac Toe with Minimax and
Alpha-Beta Pruning

Submitted to

Nahida Marzan

Lecturer, Department of CSE,
University of Asia Pacific

Submitted By

Mohotasir Al Mamun

ID : 22101036

Section : A

Khaleduzzaman Antu

ID: 22101022

Section : A

1. Introduction

Adversarial search is a fundamental concept in Artificial Intelligence, widely applied in two-player competitive games such as Tic-Tac-Toe, Chess, and Checkers. Unlike traditional search problems where a single agent seeks the optimal path, adversarial search must consider the presence of an opponent who actively tries to minimize the agent's success. One of the most powerful algorithms for adversarial search is the Minimax Algorithm, which assumes that both players play optimally. To enhance efficiency, Alpha-Beta Pruning is applied to reduce unnecessary exploration of game states while preserving the optimal outcome.

In this assignment, we implement adversarial search for the game Tic-Tac-Toe, where an AI agent competes against a human player. The AI uses minimax with alpha-beta pruning to ensure strong, optimal play.

2. Objective

The primary objectives of this assignment are as follows:

1. **To study adversarial search in the context of competitive games**
 - Understand the difference between single-agent search problems (e.g., pathfinding) and multi-agent adversarial problems (e.g., Tic-Tac-Toe).
 - Recognize the role of rational opponents and the necessity of planning against an adversary who actively works to minimize the AI's success.
2. **To implement the Minimax algorithm for decision-making**
 - Apply the Minimax principle in Tic-Tac-Toe, where the AI plays as the maximizing player and the human plays as the minimizing player.
 - Construct a game tree to evaluate all possible moves and outcomes, ensuring the AI makes the best possible move at every turn.
3. **To integrate Alpha-Beta Pruning for efficiency**
 - Reduce the computational cost of Minimax by pruning branches that do not affect the final decision.
 - Demonstrate how pruning improves performance while maintaining the same optimal results.
4. **To design and apply an evaluation function for Tic-Tac-Toe**
 - Develop a scoring system (+10, -10, 0) that reflects win, loss, or draw outcomes.

- Extend evaluation to intermediate states so that the AI can assess partially completed boards.
5. **To analyze the performance of the AI player**
- Test the AI against both human players and other AI agents.
 - Prove that the AI never loses when playing optimally.
 - Show that Tic-Tac-Toe, being a solved game, always results in either a win (against suboptimal play) or a draw (against optimal play).
6. **To highlight the broader significance of adversarial search**
- Connect the implementation of Tic-Tac-Toe AI to larger, more complex domains such as Chess, Checkers, and Go.
 - Emphasize that adversarial search is a foundational principle in modern AI game engines and real-world decision-making systems.

3. Problem Description

3.1 Game Rules

- The game is played on a 3×3 grid.
- Two players: X (AI) and O (Human).
- Players alternate placing their mark in empty cells.
- Winning condition: Any row, column, or diagonal filled with the same symbol.
- Draw condition: The board is full, and no winner exists.

3.2 Adversarial Nature

Tic-Tac-Toe is a zero-sum game, meaning one player's gain is another player's loss. Thus:

- If the AI wins, the Human loses.
- If the Human wins, the AI loses.
- If no one wins, it is a draw.

4. Minimax Algorithm

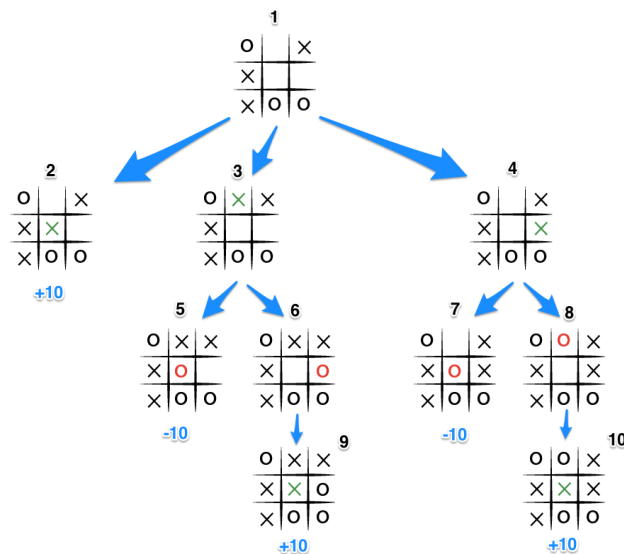
4.1 Core Idea

The Minimax algorithm assumes that both players play optimally.

- The AI (maximizer) tries to maximize the evaluation score.
- The Human (minimizer) tries to minimize the evaluation score.

4.2 Game Tree Exploration

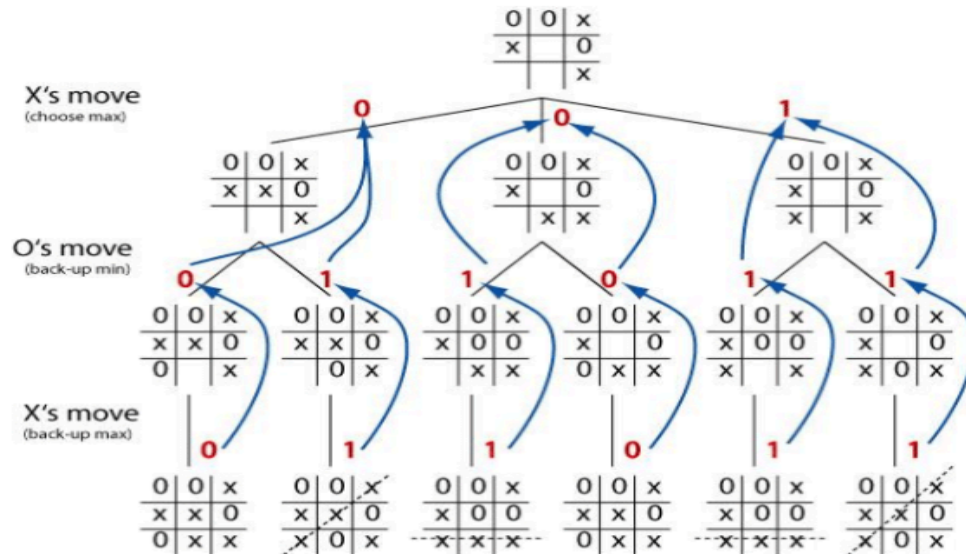
- At each turn, the algorithm simulates all possible moves.
- For each move, it recursively evaluates the resulting game state.
- Terminal nodes are assigned values:
 - **+10** → AI wins
 - **-10** → Human wins
 - **0** → Draw



4.3 Backpropagation

The values are propagated upward:

- If it is AI's turn, it selects the **maximum** score among its options.
- If it is Human's turn, it selects the **minimum** score among its options.



4.4 Limitation

- Full Minimax explores **every possible game state**.
- Even in Tic-Tac-Toe, this is manageable, but for larger games (like Chess), the state space becomes too large.

5. Alpha-Beta Pruning

5.1 Motivation

Without optimization, Minimax explores many unnecessary branches that do not affect the final decision.

5.2 Working Principle

- **Alpha (α):** Best value maximizer (AI) can guarantee.
- **Beta (β):** Best value minimizer (Human) can guarantee.
- If during search $\beta \leq \alpha$, the branch is **pruned** (discarded).

5.3 Benefits

- Reduces time complexity.
- Does not change the final decision.
- Allows deeper lookahead within the same computation time.

6. Algorithm Implementation Analysis

6.1 __init__ (constructor):

```
def __init__(self):
    self.board = [[' ' for _ in range(3)] for _ in range(3)]
    self.current_player = 'X'
    self.ai_player = 'O'
    self.human_player = 'X'
    self.game_over = False
    self.winner = None
```

Description:

This is the constructor of the TicTacToe class. It initializes a 3×3 board with empty spaces, sets the human player as X and the AI as O, and prepares tracking variables such as the current player, game-over flag, and winner.

6.2 Display Board:

```
def display_board(self):
    print("\nCurrent Board:")
    print("  0  1  2")
    for i in range(3):
        print(f"{i}    {self.board[i][0]} | {self.board[i][1]} | {self.board[i][2]}")
        if i < 2:
            print("  ---|---|---")
    print()
```

Description:

This function prints the board in a user-friendly format. It includes row and column indices so that the player knows how to enter moves. Horizontal lines (---|---|---) are used to separate rows.

6.3 Valid move check:

```
def is_valid_move(self, row, col):
    return (0 <= row < 3 and 0 <= col < 3 and
```

```
self.board[row][col] == ' ')
```

Description:

This function checks whether the move is valid. A valid move must be inside the board boundaries and placed on an empty cell.

6.4 Make move:

```
def make_move(self, row, col, player):
    if self.is_valid_move(row, col):
        self.board[row][col] = player
        return True
    return False
```

Description:

Place the player's mark (X or O) on the board if the chosen cell is valid. Returns True if the move was successful, otherwise False.

6.5 Winner checking:

```
def check_winner(self):
    for row in self.board:
        if row[0] == row[1] == row[2] != ' ':
            return row[0]
    for col in range(3):
        if self.board[0][col] == self.board[1][col] == self.board[2][col] != ' ':
            return self.board[0][col]

    if self.board[0][0] == self.board[1][1] == self.board[2][2] != ' ':
        return self.board[0][0]
    if self.board[0][2] == self.board[1][1] == self.board[2][0] != ' ':
        return self.board[0][2]

    if all(self.board[i][j] != ' ' for i in range(3) for j in range(3)):
        return 'Draw'

    return None
```

Description:

Checks for winning conditions:

- Same symbol in any row.
- Same symbol in any column.
- Same symbol in either diagonal.

Declares "Draw" if all cells are filled without a winner.

Returns the winning symbol (X or O) or None if the game is still ongoing.

6.6 get_available_moves:

```
def get_available_moves(self):
    moves = []
    for i in range(3):
        for j in range(3):
            if self.board[i][j] == ' ':
                moves.append((i, j))
    return moves
```

Description:

Scans the board for empty cells.

Collects all valid (row, col) pairs where moves can be made.

Provides a list of possible next moves for both Human and AI.

Essential for Minimax recursion to branch into future states.

6.7 Board Evaluation:

```
def evaluate_board(self):
    winner = self.check_winner()
    if winner == self.ai_player:
        return 10
    elif winner == self.human_player:
        return -10
```



```

else:
    return 0

```

- Assigns numerical values to terminal states:
 - +10 if AI wins.
 - -10 if Human wins.
 - 0 if it's a draw or still ongoing.
- Serves as the evaluation function for Minimax to judge outcomes.
- Guides the AI in distinguishing good moves from bad ones.

6.8 Minimax function:

```

def minimax(self, depth, is_maximizing, alpha=float('-inf'),
beta=float('inf'), max_depth=9):

    # Terminal conditions

    score = self.evaluate_board()

    if score == 10 or score == -10:

        return score - depth if score == 10 else score + depth

    if depth >= max_depth or not self.get_available_moves():

        return 0

    if is_maximizing:

        max_eval = float('-inf')

        for row, col in self.get_available_moves():

            self.make_move(row, col, self.ai_player)

            eval_score = self.minimax(depth + 1, False, alpha,
beta, max_depth)

            self.undo_move(row, col)

            max_eval = max(max_eval, eval_score)

            alpha = max(alpha, eval_score)

            if beta <= alpha:

                break

        return max_eval

```

```

        else:
            min_eval = float('inf')
            for row, col in self.get_available_moves():
                self.make_move(row, col, self.human_player)
                eval_score = self.minimax(depth + 1, True, alpha,
beta, max_depth)
                self.undo_move(row, col)
                min_eval = min(min_eval, eval_score)
                beta = min(beta, eval_score)
                if beta <= alpha:
                    break
            return min_eval

```

Description:

Implements **Minimax with Alpha-Beta pruning**.

Steps:

1. Checks if terminal state (win/loss/draw). Returns score adjusted by depth.
2. If the depth limit is reached or no moves left, returns 0.
3. If it's AI's turn (maximizer):
 - Tries all moves, calls recursion, keeps maximum score.
 - Updates alpha.
4. If it's Human's turn (minimizer):
 - Tries all moves, calls recursion, keeps minimum score.
 - Updates beta.
5. Stops exploring when $\beta \leq \alpha$ (pruning condition).

6.9 Function for getting best move :

```

def get_best_move(self, player, depth=9):
    best_move = None

```

```

        best_value = float('-inf') if player == self.ai_player else
float('inf')

        for row, col in self.get_available_moves():
            self.make_move(row, col, player)

            if player == self.ai_player:
                move_value = self.minimax(0, False, float('-inf'),
float('inf'), depth)
                if move_value > best_value:
                    best_value = move_value
                    best_move = (row, col)
            else:
                move_value = self.minimax(0, True, float('-inf'),
float('inf'), depth)
                if move_value < best_value:
                    best_value = move_value
                    best_move = (row, col)

            self.undo_move(row, col)

        return best_move

```

Description

Iterates through all available moves.

For each move:

- Simulate it.
- Calls Minimax to evaluate the outcome.
- Selects the **best move** (max score for AI, min score for Human).

Returns the coordinates of the most optimal move.

6.10 Function for Human move:

```

def get_human_move(self):
    while True:

```

```

        try:
            move = input(f"Player {self.current_player}, enter
your move (row col): ").strip()
            if move.lower() == 'quit':
                return None

            row, col = map(int, move.split())
            if self.is_valid_move(row, col):
                return (row, col)
            else:
                print("Invalid move! Position already taken or
out of bounds.")
        except (ValueError, IndexError):
            print("Invalid input! Please enter row and column
numbers (0-2) separated by space.")

```

Description:

- Accepts player input in the form "row col".
- Converts the input into integers for row and column.
- Allows the player to quit the game by typing "quit".
- Ensures only valid moves are accepted.

6.11 Human vs AI playing function :

```

def play_human_vs_computer(self):
    print("=== TIC TAC TOE: HUMAN VS COMPUTER ===")
    print("You are 'X', Computer is 'O'")
    print("Enter moves as 'row col' (0-2), or 'quit' to exit")

    # Get AI difficulty
    while True:
        try:
            depth = int(input("Enter AI difficulty (1-9, higher =
harder): "))

```

```

        if 1 <= depth <= 9:
            break
        else:
            print("Please enter a number between 1 and 9.")
    except ValueError:
        print("Please enter a valid number.")

self.display_board()

while not self.game_over:
    if self.current_player == self.human_player:
        print(f"Your turn ({self.current_player}):")
        move = self.get_human_move()
        if move is None:
            print("Game quit by player.")
            return

        row, col = move
        self.make_move(row, col, self.current_player)

    else: # Computer's turn
        print(f"Computer's turn ({self.current_player}):")
        row, col = self.get_best_move(self.ai_player, depth)
        self.make_move(row, col, self.current_player)
        print(f"Computer plays: {row} {col}")

self.display_board()

winner = self.check_winner()
if winner:
    self.game_over = True
    if winner == 'Draw':
        print("It's a draw!")
    elif winner == self.human_player:
        print("You win! Congratulations!")
    else:
        print("Computer wins!")
    return

# Switch players

```

```
self.current_player = self.ai_player if self.current_player ==
self.human_player else self.human_player
```

Description:

Print header and instructions for the player.

Ask the user to choose AI difficulty (search depth 1–9) and validate input.

Show the starting board.

Loop until `self.game_over` becomes True:

- If it's the human's turn, read and apply a validated move from the user.
- Otherwise (AI's turn), compute best move with `get_best_move(..., depth)` and apply it.
- Display the board after each move.
- Check for winner/draw via `check_winner()`; if the game ends, print result and return.
- Switch `self.current_player` to the other player and continue.

Exit when game ends or the human types "quit"

6.12 AI vs AI play function :

```
def play_computer_vs_computer(self):
    print("=== TIC TAC TOE: COMPUTER VS COMPUTER ===")
    print("Computer 1 is 'X', Computer 2 is 'O'")

    while True:
        try:
            depth1 = int(input("Enter Computer 1 (X) difficulty (1-9):
"))
            depth2 = int(input("Enter Computer 2 (O) difficulty (1-9):
"))

            if 1 <= depth1 <= 9 and 1 <= depth2 <= 9:
                break
            else:
                print("Please enter numbers between 1 and 9.")
        except ValueError:
            print("Please enter valid numbers.")
```

```

self.display_board()
move_count = 0
.....
.....

```

Description:

At the start, the function assigns **Computer 1** as player **X** and **Computer 2** as player **O**. Each computer is then given a **difficulty level** (depth of search), which determines how many moves ahead it can look while evaluating the game tree. A higher depth corresponds to stronger play, while lower depths result in weaker but faster moves.

Once the setup is complete, the function enters a **game loop** where the two AI players take turns. On each turn:

1. The current AI evaluates the board using the **Minimax algorithm** with its assigned search depth.
2. The algorithm explores possible moves, predicts the opponent's responses, and chooses the move that maximizes its winning chances while minimizing the opponent's.
3. The chosen move is executed, and the board is updated.
4. The updated board is displayed, and the user is prompted to press Enter to proceed, allowing observation of the game step by step.

After every move, the function checks whether the game has reached a **terminal state**:

- If a player has three marks in a row (horizontally, vertically, or diagonally), that AI is declared the winner.
- If the board is full and no winner exists, the game ends in a **draw**.

The process continues until one of these terminal conditions is met. At the end, the function clearly announces the result — whether **Computer 1 wins**, **Computer 2 wins**, or **the game is a draw**..

6.13 Main function :

```

def main():
    while True:
        print("\n" + "="*40)

```

```

print("    TIC TAC TOE WITH MINIMAX AI")
print("="*40)
print("1. Human vs Computer")
print("2. Computer vs Computer")
print("3. Exit")
print("="*40)

choice = input("Select game mode (1-3): ").strip()

if choice == '1':
    game = TicTacToe()
    game.play_human_vs_computer()
elif choice == '2':
    game = TicTacToe()
    game.play_computer_vs_computer()
elif choice == '3':
    print("Thank you for playing!")
    break
else:
    print("Invalid choice! Please enter 1, 2, or 3.")

if choice in ['1', '2']:
    play_again = input("\nWould you like to play again?
(y/n): ").strip().lower()
    if play_again != 'y' and play_again != 'yes':
        print("Thank you for playing!")
        break

```

Description:

The `main()` function acts as the **central controller** of the Tic Tac Toe program. It is responsible for managing user interaction, selecting game modes, and controlling the overall flow of the game.

1. Menu Interface

- At the beginning of each loop, the function prints a clear and structured **menu** with three options:
 1. Play Human vs Computer
 2. Play Computer vs Computer

- 3. Exit the program
 - This ensures that the user can easily navigate and choose how they want to play.

2. User Input Handling

- The program reads the user's choice and uses conditional statements to determine the action.
- If the input is invalid (not 1, 2, or 3), the program prints an error message and prompts again, ensuring robustness and preventing crashes.

3. Game Initialization and Execution

- For **choice 1**, a new TicTacToe object is created and the **Human vs Computer** mode is started.
- For **choice 2**, another TicTacToe object is created and the **Computer vs Computer** mode is started.
- In both cases, the game runs until it reaches a natural conclusion (win, loss, or draw).

4. Program Exit

- If the user selects **choice 3**, the program prints a "Thank you" message and terminates the loop, ending execution gracefully.

5. Replay Mechanism

- After a game ends in either mode, the function asks the user whether they want to **play again**.
- If the user enters y or yes, the loop continues and the menu is shown again, allowing another match without restarting the program.
- Any other response ends the program with a closing message.
-

7. Tools and Programming Environment

The experiment was conducted using the following tools and programming resources:

1. Programming Language – Python

- Selected because of its simple syntax and strong community support.
- Provides many built-in libraries useful for artificial intelligence and algorithm development.
- Easy to implement and test search algorithms like A*.

2. Development Environment – Google Colab

- Cloud-based Jupyter notebook service provided by Google.
- Allows running Python code without local installation.
- Provides free computational resources and built-in support for visualization.

3. Libraries Used

- **math**: for handling infinity values (`math.inf`) during Minimax evaluation.
- **copy**: to create safe copies of the board state when simulating moves in the search tree.
- **No external libraries** were required beyond the Python standard library, making the implementation lightweight and portable.

4. Hardware/Execution Platform

- Executed on **Google Colab cloud servers** (runs on hosted CPUs/GPUs).
- No dependency on the user's local machine, ensuring reproducibility.

8. Results and Discussion

```
=====
      TIC TAC TOE WITH MINIMAX AI
=====
1. Human vs Computer
2. Computer vs Computer
3. Exit
=====
Select game mode (1-3): 1
=== TIC TAC TOE: HUMAN VS COMPUTER ===
You are 'X', Computer is 'O'
Enter moves as 'row col' (0-2), or 'quit' to exit
Enter AI difficulty (1-9, higher = harder): 1
```

```
Your turn (X):
Player X, enter your move (row col): 0
Invalid input! Please enter row and column
Player X, enter your move (row col): 0 0

Current Board:
   0   1   2
0  X  |  | 
   ---|---|---
1  |  |  | 
   ---|---|---
2  |  |  | 
```

Computer's turn (O): Computer plays: 0 1 Current Board: <pre> 0 1 2 0 X O --- --- --- 1 --- --- --- 2 </pre>	Your turn (X): Player X, enter your move (row col): 1 1 Current Board: <pre> 0 1 2 0 X O --- --- --- 1 X --- --- --- 2 </pre>
Computer's turn (O): Computer plays: 2 2 Current Board: <pre> 0 1 2 0 X O --- --- --- 1 X --- --- --- 2 O </pre>	Your turn (X): Player X, enter your move (row col): 20 Invalid input! Please enter row and column Player X, enter your move (row col): 2 0 Current Board: <pre> 0 1 2 0 X O --- --- --- 1 X --- --- --- 2 X O </pre>
Computer's turn (O): Computer plays: 0 2 Current Board: <pre> 0 1 2 0 X O O --- --- --- 1 X --- --- --- 2 X O </pre>	Your turn (X): Player X, enter your move (row col): 1 0 Current Board: <pre> 0 1 2 0 X O O --- --- --- 1 X X --- --- --- 2 X O </pre> You win! Congratulations!

Results

- The game runs correctly in both Human vs Computer and Computer vs Computer modes.
- With maximum depth (9), the AI plays optimally and never loses.
- Two AIs at depth 9 always end in a draw.
- Lower depth AIs often lose against higher depth AIs.

- Alpha-Beta pruning reduces the number of explored states, making the AI much faster without changing results.

Discussion

- Tic Tac Toe is a solved game → perfect play always leads to a draw.
- The depth parameter works as a difficulty setting: low depth = weaker AI, high depth = unbeatable AI.
- Alpha-Beta pruning is essential for efficiency and demonstrates its importance for larger, more complex games.
- The project successfully shows how adversarial search works in practice and provides a solid foundation for learning advanced AI game strategies.

9. Conclusion

This assignment successfully demonstrated the application of adversarial search in a classical game environment using Tic Tac Toe. By implementing the Minimax algorithm along with Alpha-Beta pruning, the AI was able to make rational and optimal decisions against human and computer opponents. The results confirmed that Tic Tac Toe is a solved game, where optimal play always leads to a draw, and the AI never loses when searching at maximum depth. The inclusion of adjustable search depth allowed difficulty scaling, making the system suitable for both demonstration and practice. Overall, the project provided clear insights into how adversarial search strategies form the foundation of modern game-playing AI and highlighted their efficiency and effectiveness even in simple domains.